STATSAI

AI for Data Science & Statistics

Module 1

Programming Foundations for Data Science in Python & R

A 2–3 hour lecture

Phase A — Foundations

Yousif Alyousifi, PhD · University of Michigan

About this Lecture

This is the first lecture of the StatsAI program. By the end of this 2-3 hour session, you will have a working Python and R environment, understand the core data structures of both languages, and have written your first analytical code in both.

This lecture assumes you have never written serious data analysis code before, but you are comfortable with basic computer use. We will move at a steady pace — we cover a lot, but every concept is grounded in something you will type and run yourself.

Learning Objectives

After this lecture, you will be able to:

1. Install and verify a working Python environment (Anaconda or uv) and R/RStudio.
2. Import data from a CSV file in both Python and R.
3. Manipulate the most common data structures: lists, dictionaries, NumPy arrays, pandas DataFrames (Python); vectors, lists, data frames, tibbles (R).
4. Write functions, use control flow (if/for/while), and handle missing values.
5. Initialize a Git repository and make your first commit.
6. Recognize when to reach for Python vs. R for a given task.

Prerequisites

- Basic computer literacy (file system, terminal/command prompt).
- College-level math (no calculus needed for this module).
- A laptop with at least 8 GB RAM and 10 GB free disk space.
- An internet connection for downloads.

How to use this lecture

Read sequentially. Type every code example yourself — do not copy-paste. The whole point of this module is muscle memory, and you only get that by typing. After each major section there are exercises; do them before moving on.

> ⏱️ **Time budget**
>
> Setup (40 min) · Python core (40 min) · R core (40 min) · Data structures comparison (20 min) · Git basics (15 min) · Exercises (25 min). Total: ~3 hours.

1. Environment Setup

Before we write any code, we need to install Python, R, and a few essential tools. Follow the instructions for your operating system. Skip ahead to whichever you use.

1.1 Why two languages?

Python and R are the two dominant languages in data science. Many programs teach only one. We teach both because they have different strengths, and the best practitioners are bilingual:

- Python excels at machine learning, deep learning, production deployment, and integration with software systems. The AI/LLM ecosystem (transformers, LangChain, OpenAI, Anthropic SDKs) lives almost entirely in Python.
- R excels at statistical modeling, exploratory analysis, and visualization. Most cutting-edge statistical methods (mixed models, survival analysis, Bayesian methods) get implemented in R first.
- In this program, you will see Python in the AI modules (5–8) and a mix of R and Python in the foundations and statistics modules (1–4). The capstone (Module 10) is your choice.

1.2 Installing Python

We recommend Anaconda (the data-science-friendly Python distribution) for beginners, or uv (a modern, fast alternative) for those comfortable with the command line.

Option A: Anaconda (recommended for most students)

7. Visit anaconda.com/download and download the installer for your OS (Windows, macOS, or Linux).
8. Run the installer. Accept the defaults except: when asked, check "Add Anaconda to my PATH environment variable" if you're on Windows.
9. Open a new terminal (Anaconda Prompt on Windows; Terminal on macOS/Linux) and verify with:

```shell
python --version
# Expected output: Python 3.11.x or similar (anything 3.10+)

conda --version
# Expected output: conda 24.x.x or similar
```

Option B: uv (fast and modern)

If you're comfortable in the terminal, uv is significantly faster than conda and is the modern standard among Python developers in 2026:

```shell
# macOS/Linux:
```

```
curl -LsSf https://astral.sh/uv/install.sh | sh

# Windows (PowerShell):
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"

# Then verify:
uv --version
uv python install 3.12
```

1.3 Installing R and RStudio

R is the statistical language. RStudio is the most popular editor for R — install both.

10. Download R from cran.r-project.org. Pick your OS, then download the latest base R.
11. Run the installer with default settings.
12. Download RStudio Desktop (Free) from posit.co/download/rstudio-desktop. Install it.
13. Open RStudio. In the bottom-left console, type:

```R
R.version.string
# Expected output: "R version 4.4.x ..." or similar (anything 4.0+)

1 + 1
# Should print: [1] 2
```

1.4 Installing essential packages

Python and R each have a few core packages we will use throughout the program. Install them now so we don't have to wait later.

Python packages

```SHELL
# In your terminal:
pip install numpy pandas matplotlib seaborn jupyter scikit-learn

# Or with uv:
uv pip install numpy pandas matplotlib seaborn jupyter scikit-learn

# Verify by starting Python and importing:
python -c "import numpy, pandas, matplotlib, seaborn, sklearn; print('All
good')"
```

R packages

```R
# In the RStudio console:
```

```
install.packages(c("tidyverse", "data.table", "ggplot2", "readr",
"dplyr"))

# This will take a few minutes the first time.
# When done, verify:
library(tidyverse)
# Should print version info and any conflicts (conflicts are expected, not
errors)
```

> 💡 **Tip**
>
> Package installation can take 5-15 minutes depending on your internet speed. Run these commands now and let them finish in the background while you read the next section.

2. Python Core

Let's start with Python. Open a Jupyter Notebook (run jupyter notebook in your terminal, or use VS Code with the Python extension). Type each example yourself.

2.1 Variables and types

Python uses dynamic typing — you assign a value to a name and Python figures out the type:

PYTHON

```
# Numbers
age = 35
weight_kg = 78.5

# Strings
name = "Yousif"
country = 'Yemen'   # single or double quotes — both work

# Booleans
is_researcher = True
is_retired = False

# None (Python's null)
spouse = None

# Check the type
print(type(age))           # <class 'int'>
print(type(weight_kg))     # <class 'float'>
print(type(name))          # <class 'str'>
print(type(is_researcher)) # <class 'bool'>
```

2.2 The four core data structures

These four are the building blocks of every Python data analysis. Master them now and the rest gets much easier.

Lists — ordered, mutable sequences

PYTHON

```
ages = [35, 42, 28, 51, 39]

# Indexing (zero-based)
print(ages[0])    # 35
print(ages[-1])   # 39 (negative indexes from the end)

# Slicing
print(ages[1:4])  # [42, 28, 51]

# Modify
```

```
ages.append(45)        # add to end
ages[0] = 36           # change in place
ages.remove(28)        # remove first occurrence
print(len(ages))       # current length

# Useful operations
print(min(ages), max(ages), sum(ages))
print(sorted(ages))
```

Dictionaries — key-value pairs

PYTHON

```
patient = {
    "id": "P001",
    "age": 54,
    "diagnosis": "MASH",
    "alt": 87.2,
    "on_treatment": True
}

# Access by key
print(patient["age"])           # 54
print(patient.get("weight"))    # None (no error if missing)

# Add or modify
patient["weight"] = 78.5
patient["age"] = 55

# Iterate
for key, value in patient.items():
    print(f"{key}: {value}")
```

NumPy arrays — fast numeric computing

PYTHON

```
import numpy as np

# Create an array from a list
alt_values = np.array([45.2, 87.1, 92.3, 38.7, 110.5])

# Vectorized operations (much faster than loops)
print(alt_values * 2)            # multiply every element
print(alt_values.mean())         # 74.76
print(alt_values.std())          # 27.46 (standard deviation)
print(alt_values > 40)           # boolean array: [True, True, ...]

# 2D arrays (matrices)
X = np.array([[1, 2, 3], [4, 5, 6]])
print(X.shape)   # (2, 3)
print(X.T)       # transpose
```

```
# Useful constructors
zeros = np.zeros(10)              # [0, 0, ...]
linspace = np.linspace(0, 1, 5)   # [0, 0.25, 0.5, 0.75, 1.0]
random_norm = np.random.normal(0, 1, 100)  # 100 samples from N(0,1)
```

pandas DataFrames — the workhorse of data analysis

PYTHON

```
import pandas as pd

# Create a DataFrame from a dictionary
df = pd.DataFrame({
    "patient_id": ["P001", "P002", "P003", "P004"],
    "age":        [54, 62, 47, 71],
    "alt":        [87.2, 110.5, 38.7, 92.3],
    "diagnosis":  ["MASH", "MASH", "NAFLD", "MASH"]
})

# Inspect
print(df.head())
print(df.shape)         # (4, 4)
print(df.dtypes)        # column types
print(df.describe())    # numeric summary

# Select columns
print(df["age"])              # one column (a Series)
print(df[["age", "alt"]])      # multiple columns (a DataFrame)

# Filter rows
high_alt = df[df["alt"] > 80]
print(high_alt)

# Add a new column
df["alt_high"] = df["alt"] > 80

# Group by and summarize
df.groupby("diagnosis")["alt"].mean()
```

> 📌 **Why DataFrames matter**
>
> A DataFrame is essentially a spreadsheet that you can manipulate with code. Almost every data analysis you ever do in Python will start by loading data into a DataFrame. Spend extra time here — it pays off forever.

2.3 Control flow

Conditionals and loops are how you make decisions and process data:

PYTHON

```
# if / elif / else
alt = 87.2
if alt > 100:
    severity = "severe"
elif alt > 60:
    severity = "moderate"
elif alt > 40:
    severity = "mild"
else:
    severity = "normal"
print(severity)

# for loop
patients = ["P001", "P002", "P003"]
for pid in patients:
    print(f"Processing {pid}")

# Range-based for loop
for i in range(5):
    print(i)   # 0, 1, 2, 3, 4

# while loop (use sparingly — for is usually clearer)
n = 10
while n > 0:
    print(n)
    n -= 1

# List comprehensions (a Python superpower)
alt_values = [45, 87, 110, 38, 92]
high_alt = [x for x in alt_values if x > 80]
print(high_alt)   # [87, 110, 92]

# Squared values
squared = [x**2 for x in alt_values]
```

2.4 Functions

A function is a reusable block of code. As soon as you write the same logic twice, wrap it in a function:

PYTHON

```
def classify_alt(value):
    """Classify ALT value into severity buckets."""
    if value > 100:
        return "severe"
    elif value > 60:
        return "moderate"
    elif value > 40:
        return "mild"
    else:
```

```
        return "normal"

# Use it
print(classify_alt(87.2))  # moderate

# Apply it to a whole DataFrame column
df["severity"] = df["alt"].apply(classify_alt)

# Functions with multiple arguments and defaults
def normalize(x, mean=0, std=1):
    return (x - mean) / std

print(normalize(85, mean=70, std=15))  # 1.0
```

2.5 Reading a CSV file

This is the most common operation you'll ever do. Download a sample CSV (we provide one) and read it:

PYTHON

```
# Read a CSV file from disk
df = pd.read_csv("liver_data.csv")

# Or directly from a URL
url = "https://raw.githubusercontent.com/example/data/main/liver.csv"
df = pd.read_csv(url)

# Useful options
df = pd.read_csv(
    "liver_data.csv",
    sep=",",              # column separator
    header=0,             # row index of the header (0 = first row)
    na_values=["NA", "", "-999"],  # strings to treat as missing
    parse_dates=["date"], # columns to parse as dates
    nrows=1000            # only read first 1000 rows (useful for big
files)
)

# Save back to CSV
df.to_csv("liver_clean.csv", index=False)
```

2.6 Handling missing values

Real data is messy. Missing values (represented as NaN in pandas) are everywhere. You must learn to handle them deliberately:

PYTHON

```
# Check for missing values
```

```
print(df.isna().sum())          # count per column
print(df.isna().any(axis=1))    # True for rows with any missing

# Drop rows with any missing values
df_clean = df.dropna()

# Drop rows with missing values in specific columns only
df_clean = df.dropna(subset=["age", "alt"])

# Fill missing values
df["age"] = df["age"].fillna(df["age"].mean())   # fill with mean
df["diagnosis"] = df["diagnosis"].fillna("unknown") # fill with constant
```

> ⚠️ **Important**
>
> Never silently drop missing values without understanding them first. Missing values often carry information (which patients didn't have a test? which days had broken sensors?). Always investigate the pattern of missingness before deciding how to handle it. We will revisit this in Module 2.

3. R Core

Now we cover the same concepts in R. Open RStudio. Create a new R script (File → New File → R Script). Type each example.

3.1 Why R is different

R was designed by statisticians for statisticians. This means a few things:

- R has vectors as first-class citizens — operations are vectorized by default.
- R uses 1-based indexing (the first element is x[1], not x[0]). This catches Python users off guard.
- R uses <- for assignment (though = also works). The convention is <-.
- R has no "main" data frame library by default — but the tidyverse ecosystem has become the standard.

3.2 Variables and basic types

R

```
# Assignment uses <-
age <- 35
weight_kg <- 78.5
name <- "Yousif"
is_researcher <- TRUE   # uppercase! TRUE/FALSE, not True/False

# Print
print(age)
age              # in interactive mode, just typing the name prints it

# Check the type/class
class(age)         # "numeric"
class(name)        # "character"
class(is_researcher)  # "logical"

# R-specific: there is also integer and double
x <- 5L            # integer (the L makes it integer)
y <- 5.0           # double (default for numbers)
```

3.3 Vectors — R's fundamental data structure

In R, a single number is actually a vector of length 1. Vectors are ordered, homogeneous (all elements same type), and the basis of everything:

R

```
# Create a vector with c() ("combine")
ages <- c(35, 42, 28, 51, 39)
```

```
# Indexing (1-based!)
ages[1]            # 35 (first element)
ages[length(ages)] # 39 (last element)
ages[2:4]          # [42, 28, 51] — slicing
ages[ages > 35]    # [42, 51, 39] — boolean filtering

# Vectorized operations
ages * 2           # multiplies every element
mean(ages)         # 39
sd(ages)           # standard deviation
summary(ages)      # min, 1Q, median, mean, 3Q, max

# Generate sequences
1:10               # 1, 2, 3, ..., 10
seq(0, 1, by=0.1)  # 0, 0.1, 0.2, ..., 1.0
rnorm(100)         # 100 samples from N(0,1)
rep("A", 5)        # "A" "A" "A" "A" "A"
```

3.4 Lists — heterogeneous collections

Unlike vectors, lists can hold different types of elements. They are R's equivalent of Python dictionaries:

R

```
patient <- list(
  id = "P001",
  age = 54,
  diagnosis = "MASH",
  alt = 87.2,
  on_treatment = TRUE
)

# Access by name
patient$age          # 54  (preferred)
patient[["age"]]     # 54  (also works)
patient["age"]       # returns a sub-list, not the value!

# Add or modify
patient$weight <- 78.5
patient$age <- 55

# Get all names
names(patient)
```

3.5 Data frames — R's spreadsheet

Data frames are the R equivalent of pandas DataFrames. The tidyverse uses an enhanced version called a tibble, which we strongly recommend:

R

```
library(tidyverse)   # loads dplyr, tibble, ggplot2, and friends

# Create a tibble
df <- tibble(
  patient_id = c("P001", "P002", "P003", "P004"),
  age        = c(54, 62, 47, 71),
  alt        = c(87.2, 110.5, 38.7, 92.3),
  diagnosis  = c("MASH", "MASH", "NAFLD", "MASH")
)

# Inspect
print(df)            # nice formatted print
head(df)             # first 6 rows
dim(df)              # rows, cols
str(df)              # structure
summary(df)          # numeric summary

# Select columns (the dplyr way)
df %>% select(age, alt)

# Filter rows
df %>% filter(alt > 80)

# Add a new column
df %>% mutate(alt_high = alt > 80)

# Group by and summarize
df %>%
  group_by(diagnosis) %>%
  summarise(mean_alt = mean(alt))
```

📌 The pipe (%>%)

The %>% operator (pronounced "then") chains operations together: take the result of the left side and pass it to the right. This makes code read naturally — "take df, then filter, then group, then summarize." R 4.1+ also supports |> as a built-in pipe with the same effect.

3.6 Control flow in R

R

```
# if / else if / else
alt <- 87.2
if (alt > 100) {
  severity <- "severe"
} else if (alt > 60) {
  severity <- "moderate"
} else if (alt > 40) {
  severity <- "mild"
} else {
  severity <- "normal"
```

```
}
print(severity)

# for loop
patients <- c("P001", "P002", "P003")
for (pid in patients) {
  print(paste("Processing", pid))
}

# But often you don't need a loop — vectorize instead!
alt_values <- c(45, 87, 110, 38, 92)
high_alt <- alt_values[alt_values > 80]
print(high_alt)   # [87, 110, 92]

# Or use ifelse() for vectorized conditionals
severity <- ifelse(alt_values > 80, "high", "normal")
print(severity)
```

3.7 Functions in R

R

```
classify_alt <- function(value) {
  if (value > 100) {
    return("severe")
  } else if (value > 60) {
    return("moderate")
  } else if (value > 40) {
    return("mild")
  } else {
    return("normal")
  }
}

# Use it
classify_alt(87.2)  # "moderate"

# Apply to a whole column
df %>% mutate(severity = sapply(alt, classify_alt))

# Or vectorize properly with case_when()
df %>%
  mutate(severity = case_when(
    alt > 100 ~ "severe",
    alt > 60  ~ "moderate",
    alt > 40  ~ "mild",
    TRUE      ~ "normal"
  ))
```

3.8 Reading a CSV file in R

```R
# Using readr (part of tidyverse)
df <- read_csv("liver_data.csv")

# Or from a URL
url <- "https://raw.githubusercontent.com/example/data/main/liver.csv"
df <- read_csv(url)

# Useful options
df <- read_csv(
  "liver_data.csv",
  na = c("NA", "", "-999"),       # strings to treat as missing
  col_types = cols(                # explicit column types
    age = col_integer(),
    alt = col_double(),
    date = col_date()
  ),
  n_max = 1000                     # read first 1000 rows only
)

# Save back
write_csv(df, "liver_clean.csv")
```

3.9 Missing values in R

R uses NA for missing values. Most R functions require explicit handling:

```R
# Check for missing values
is.na(df$age)             # logical vector
sum(is.na(df$age))        # count of missing
colSums(is.na(df))        # missing per column

# Drop missing rows
df_clean <- df %>% drop_na()                    # any column
df_clean <- df %>% drop_na(age, alt)            # specific columns

# Fill missing values
df <- df %>% mutate(age = ifelse(is.na(age), mean(age, na.rm=TRUE), age))

# A WARNING that catches everyone:
# Most R statistical functions need na.rm=TRUE explicitly
mean(c(1, 2, NA, 4))           # NA   (the default!)
mean(c(1, 2, NA, 4), na.rm=TRUE)  # 2.33
```

4. Python vs R — Side by Side

Now that you've seen both, let's compare the same operations directly. This is the cheat sheet to come back to whenever you switch contexts:

Operation	Python	R
Assign value	`x = 5`	`x <- 5`
Length of vector	`len(arr)`	`length(v)`
First element	x[0]	x[1]
Mean of column	`df['x'].mean()`	`mean(df$x)`
Filter rows	`df[df['x']>5]`	`df %>% filter(x>5)`
Group + summarize	`df.groupby('g')['x'].mean()`	`df %>% group_by(g) %>% summarise(m=mean(x))`
Read CSV	`pd.read_csv('f.csv')`	`read_csv('f.csv')`
Missing value	`None / np.nan`	`NA`
Drop missing rows	`df.dropna()`	`df %>% drop_na()`
Linear regression	`smf.ols('y~x',df).fit()`	`lm(y~x, data=df)`
Plot scatter	`plt.scatter(x,y)`	`ggplot(df,aes(x,y))+geom_point()`

4.1 When to use which

There's no universal answer, but here's the practical guidance from working in both languages for years:

Reach for Python when:

- You're building or using machine learning models, especially deep learning.
- You're working with the AI/LLM ecosystem (transformers, LangChain, OpenAI/Anthropic SDKs).
- You need to integrate with web services, APIs, or production systems.
- You're processing large amounts of unstructured data (text, images, audio).

Reach for R when:

- You're doing classical statistics: hypothesis testing, mixed models, survival analysis.
- You need publication-quality statistical visualizations (ggplot2 is unmatched).
- You're using cutting-edge statistical methods (most appear in R first).
- You're producing reproducible reports (RMarkdown and Quarto are excellent).

Use both when:

- You're doing serious applied statistics + ML — which is most of what we'll do in this program.
- Different team members prefer different languages — interoperability is the norm now (reticulate in R, rpy2 in Python).

5. Git and GitHub Basics

If you remember one tool from this lecture besides the languages themselves, make it Git. Git is version control — it tracks every change you make to your code so you can: undo mistakes, collaborate, and (this is the big one) prove what you did and when. For research, this is non-negotiable.

5.1 Install and configure Git

SHELL

```
# macOS — Git comes with Xcode Command Line Tools:
xcode-select --install

# Windows — install from git-scm.com/download/win

# Linux:
sudo apt install git    # Debian/Ubuntu

# After installing, set your identity ONCE:
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

# Verify
git --version
```

5.2 The core Git workflow

Every project follows the same pattern. Memorize this:

SHELL

```
# Step 1: Create a folder for your project
mkdir my-analysis
cd my-analysis

# Step 2: Initialize Git
git init

# Step 3: Add some files (e.g., your Python or R script)
echo "# My Analysis" > README.md
echo "print('hello')" > analysis.py

# Step 4: See what changed
git status

# Step 5: Stage the files for commit
git add README.md analysis.py
# Or stage everything:
git add .

# Step 6: Commit with a descriptive message
```

```
git commit -m "Initial commit: project skeleton"

# Step 7: Look at history
git log --oneline
```

5.3 Connecting to GitHub

GitHub is a free hosting service for Git repositories. To put your project on GitHub:

14. Create a free account at github.com.
15. On GitHub, click "+" → "New repository". Give it a name. Don't add any files yet.
16. Copy the URL (looks like https://github.com/yourname/my-analysis.git).
17. Run these commands to push your local code to GitHub:

SHELL

```
git remote add origin https://github.com/yourname/my-analysis.git
git branch -M main
git push -u origin main

# Going forward, after each commit, just:
git push
```

> 💡 **The most useful Git habit**
>
> Commit early, commit often, with descriptive messages. "Fix bug" is bad. "Fix off-by-one error in age binning function" is good. Future-you will thank present-you in a month when you need to find when something changed.

5.4 The .gitignore file

Some files should never be in version control (data files, large binaries, secrets). Create a .gitignore file in your project root listing patterns to skip:

SHELL

```
# .gitignore for a typical data science project

# Data files (sensitive or large)
*.csv
*.xlsx
data/
raw_data/

# Outputs
*.pdf
outputs/
figures/

# Python
```

```
__pycache__/
*.pyc
.ipynb_checkpoints/
venv/
.venv/

# R
.Rhistory
.RData
.Rproj.user/

# IDE
.vscode/
.idea/

# Secrets
.env
*.key
```

6. Exercises

Now you put it all together. Do these exercises. Don't read the next module until you've solved at least the first three. The point is muscle memory — the only way to build it is to type.

> 📥 **Sample data**
>
> For these exercises, use the sample dataset at: https://github.com/Abo85Mustafa/StatsAI/blob/main/docs/data/liver_sample.csv (you'll create this file in Module 2). For now, generate synthetic data using the code in each exercise.

Exercise 1 — Basic data manipulation in Python

Write a Python script that:

18. Creates a DataFrame with 100 synthetic patients (use np.random).
19. Each patient has: id (P001..P100), age (uniform 30-80), alt (normal mean=50, sd=20), diagnosis (random from {MASH, NAFLD, normal}).
20. Computes the mean ALT for each diagnosis group.
21. Adds a column severity that classifies ALT into the four buckets we defined earlier.
22. Saves the result to patients.csv.

Starter code

PYTHON

```
import numpy as np
import pandas as pd

np.random.seed(42)   # reproducibility
n = 100

df = pd.DataFrame({
    "id": [f"P{i:03d}" for i in range(1, n+1)],
    "age": np.random.randint(30, 81, n),
    "alt": np.random.normal(50, 20, n).clip(min=5),
    "diagnosis": np.random.choice(["MASH", "NAFLD", "normal"], n)
})

# Your code here:
# 1. Compute mean ALT per diagnosis
# 2. Add severity column
# 3. Save to CSV
```

Exercise 2 — The same task in R

Now write the R equivalent of Exercise 1. The output should be identical.

Starter code

```r
library(tidyverse)
set.seed(42)
n <- 100

df <- tibble(
  id = sprintf("P%03d", 1:n),
  age = sample(30:80, n, replace = TRUE),
  alt = pmax(rnorm(n, 50, 20), 5),
  diagnosis = sample(c("MASH", "NAFLD", "normal"), n, replace = TRUE)
)

# Your code here:
# 1. Compute mean ALT per diagnosis using group_by + summarise
# 2. Add severity column using case_when
# 3. Save to CSV with write_csv
```

Exercise 3 — Functions

Write a function in BOTH Python and R that takes a numeric vector and returns a dictionary/list with: count, mean, median, sd, min, max, n_missing. Test it on a vector with at least one NA/None value.

Exercise 4 — Real CSV

Find a real CSV file on the internet (try data.gov, kaggle.com, or your own research data — anything with at least 5 columns and 100 rows). In both Python and R:

23. Read it in.
24. Print the first 10 rows, the column types, and a summary.
25. Identify any columns with missing values and report the count.
26. Pick a numeric column and compute its mean, median, and standard deviation.
27. Pick a categorical column and compute the count per category.

Exercise 5 — Git

Combine everything you've learned:

28. Create a folder called statsai-module1.
29. Initialize Git in it.
30. Add a README.md, your Python solution, and your R solution from the exercises above.
31. Make at least 3 commits with descriptive messages (one per exercise, for example).
32. Push it to a new public repo on GitHub.
33. Send the URL to the program — this is your first deliverable.

7. Recap and What's Next

What you should be able to do now

After completing this lecture and the exercises, check that you can:

- ✓ Open a Jupyter Notebook and an RStudio session and write working code in both.
- ✓ Read a CSV file in Python and R.
- ✓ Filter, select, and summarize data with pandas in Python and dplyr in R.
- ✓ Write a function and apply it to a column.
- ✓ Recognize and handle missing values.
- ✓ Initialize a Git repo, commit changes, and push to GitHub.

If any of these feel shaky, redo the relevant section and the corresponding exercise. The next module assumes all of this is comfortable.

Common pitfalls to avoid

- Mixing up Python's 0-based and R's 1-based indexing. Yes, it will catch you. Slow down when switching.
- Using = vs <- in R. Stick to <- for assignment to maintain consistency with R style guides.
- Forgetting na.rm=TRUE in R statistical functions. Most return NA when NA is present, by design.
- Mutating data without saving the result. df.dropna() in Python returns a new DataFrame; the original is unchanged. Same for most R operations.

Reading list (optional but recommended)

- Wes McKinney — "Python for Data Analysis" (3rd ed). The pandas creator's book.
- Hadley Wickham & Garrett Grolemund — "R for Data Science" (2nd ed). Free online at r4ds.hadley.nz.
- Scott Chacon — "Pro Git" (2nd ed). Free online at git-scm.com/book.

Module 2 preview

In Module 2 we go deeper on data wrangling and exploratory data analysis (EDA). You'll learn how to clean messy real-world data, detect outliers, build the right visualizations to ask the right statistical questions, and tell a coherent story from a dataset. We assume everything from Module 1, especially comfortable manipulation of pandas DataFrames and dplyr tibbles.

End of Module 1.